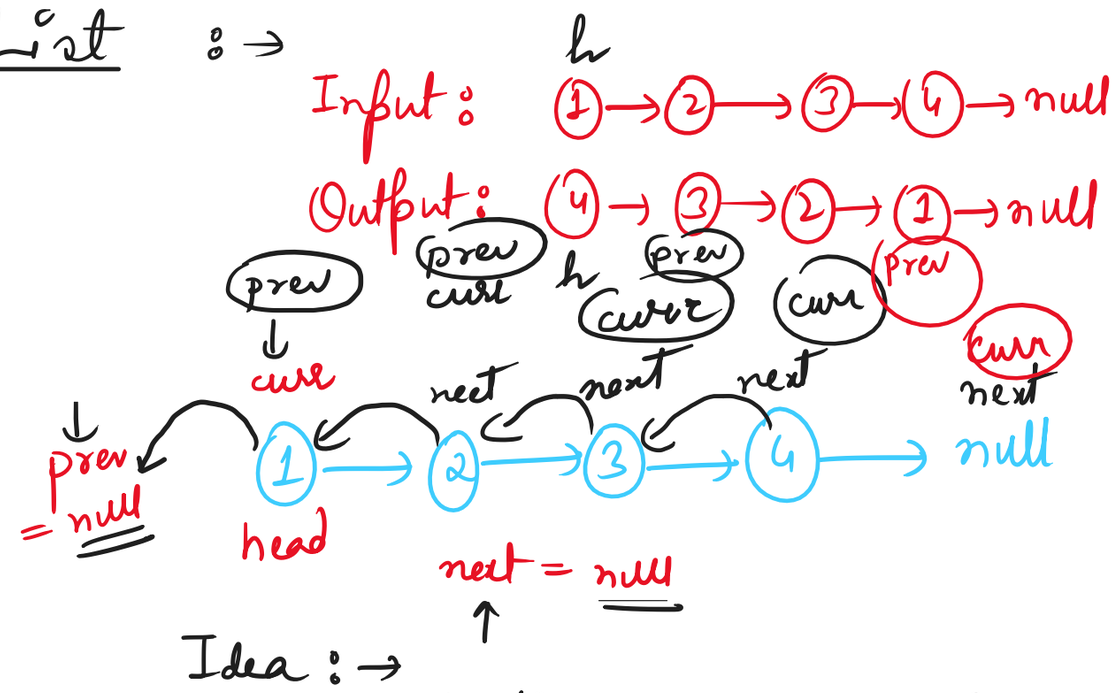
* Reverse a Linked List :→

Input: 1 → 2 → 3 → 4 → null

Output: 4 → 3 → 2 → 1 → null

Three pointers :→

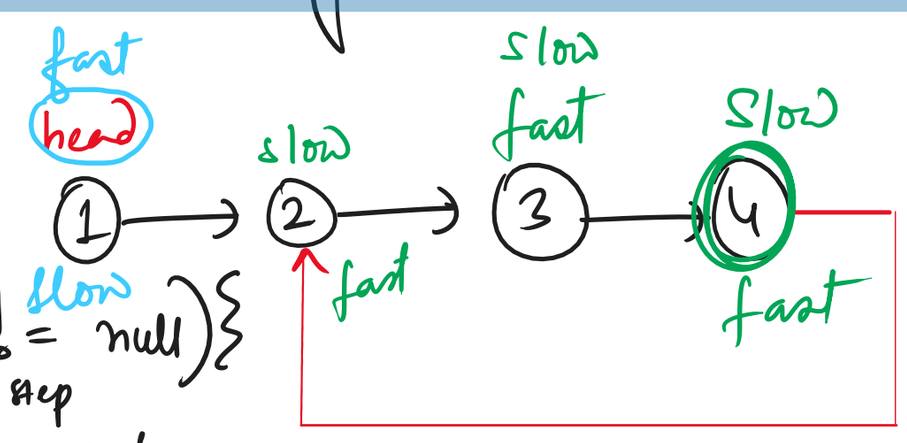
```
Node prev = null;
Node curr = head;
Node next = null;
While (curr != null) {
  ① next = curr.next;
  ② curr.next = prev;
     prev = curr;
     curr = next;
}
return prev, (new head of the list)
```



Idea :→
① Hold the remaining nodes.
② Change direction of current node.

Floyd's Cycle Detection Algorithm or Hare & Tortoise Algorithm :→

```
boolean hasCycle(head){
  Node slow = head;
  Node fast = head;
  While (fast != null && fast.next != null){
    ① slow = slow.next;      → 1 step
    ② fast = fast.next.next; → 2 step
  }  if (slow == fast)
       return true;
  return false;
}
```



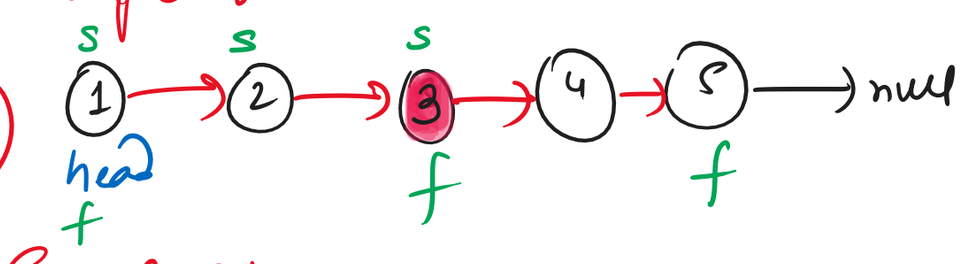
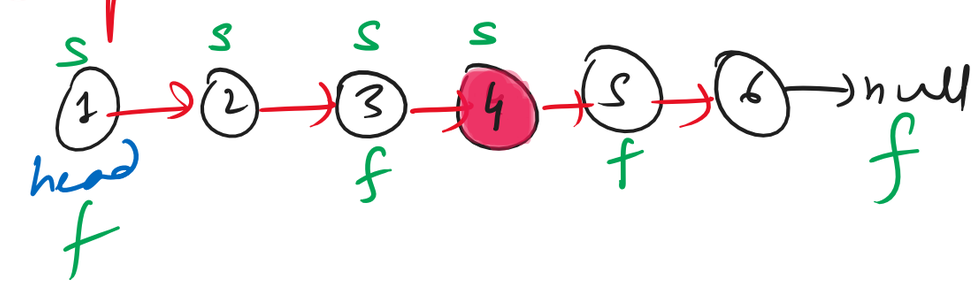
Two pointers: →
slow → 1 step
fast → 2 steps

Bonus Question: Repeated Multiple Times in Interviews

* Find the middle node of a Linked List :→ (Floyd's Algorithm)

```
Node slow = head;
Node fast = head;
while (fast != null && fast.next != null)
{
    slow = slow.next;      1 step
    fast = fast.next.next; 2 steps
}
return slow;
```

Example 1:



Example 2: